Project Reset Report — flyerrr

Deliverable 1: Project Reset & Architecture Baseline

Project	flyerrr — campus event discovery & attendance
Repository	`github.com/oanyamele/flyerrr` (private)
Audited commit	`f0c475a` on `main`
Audit date	August 27, 2026 · re-verified September 1, 2026 (HEAD unchanged)
Team	oanyamele · neaylasimone · nadcaesar · Brian-Two
Origin	Lovable.dev generated scaffold, extended by hand

Executive Summary

flyerrr is a **visually complete, functionally hollow** prototype. All 14 routes render, the design system is coherent across light and dark themes, and the production build succeeds in 2.09 s with zero TypeScript errors. That is a real asset and it should be kept.

The problem is underneath the surface. The application has **no backend of any kind** — there is not a single `fetch()` call in 7,600 lines of source. Everything persists to `window.localStorage` on one device. Because of that, the product's central mechanic — a student presenting a QR code that an *organizer on a different device* scans — **cannot work as designed, and no amount of polish will make it work.** Tracing that flow end to end (§3, R-01/R-02) shows attendance is never recorded anywhere, and all four screens that display attendance, points, and badges read hardcoded arrays.

Senior Project II therefore does not begin with new features. It begins with **building the server tier that the prototype has been faking**, and with **converting a draft-note project board into a traceable, estimated backlog**.

Readiness scorecard

Dimension	Rating	One-line justification
UI / design system	🟢 **Strong**	Coherent token system, dark mode, responsive, mobile-first
Build & typecheck	🟢 **Strong**	`vite build` clean; `tsc --noEmit` passes
Feature completeness (visual)	🟢 **Strong**	14 routes, both user roles, all screens render
Architecture	🔴 **Critical gap**	No backend, no service seam, pages import storage directly
Core feature correctness	🔴 **Critical gap**	Check-in → attendance → rewards chain is fully severed
Security	🔴 **Critical gap**	Plaintext passwords, bypassable auth, forgeable QR tokens
Test coverage	🔴 **Critical gap**	1 test, and it asserts `true === true`; E2E config broken
Engineering process	🟠 **Weak**	No CI, no branch protection, 0 Issues, 0 milestones
Backlog quality	🟠 **Weak**	Board exists but holds draft notes only — see §4
Documentation	🔴 **Critical gap**	README is literally `TODO: Document your project here`

---

1. Method

Findings are evidence-based, not impressions. Every claim below was produced by executing something against the repository at `f0c475a`:

```
bun install                      # 491 packages
npx tsc --noEmit -p tsconfig.app.json   # → 0 errors
npm run build                    # → built in 2.09s
npm test                         # → 1 passed
npx eslint .                     # → 16 problems (5 errors, 11 warnings)
npm audit --production           # → 11 vulnerabilities (10 high, 1 moderate)
gh issue list --state all        # → (empty)
gh api repos/oanyamele/flyerrr/actions/workflows  # → total_count: 0
gh api repos/oanyamele/flyerrr/milestones         # → 0
gh api repos/oanyamele/flyerrr/branches/main/protection  # → 404 Not Found
```

Dead-code and usage analysis was done by cross-referencing every component against its import sites across `src/`.

---

2. Current System Architecture

Full diagrams — system architecture, check-in sequence, persistence topology, and the proposed target architecture — are maintained in **`docs/architecture.md`** and render natively on GitHub.

2.1 Component inventory

Layer	Location	Count	Lines
Route components	`src/pages/`	15	1,802
Shared components	`src/components/`	7	574
Design-system components	`src/components/ui/`	48	3,954
React contexts	`src/contexts/`	3	134
Data access	`src/data/`	4	761
Hooks / utils	`src/hooks/` , `src/lib/`	3	211
Total source	`src/`	**80**	**≈ 7,623**

2.2 Data flow

```
Route component ──► React Context (auth/follow/theme) ──► localStorage
       │
       └──────────► src/data/*.ts (appDatabase, checkInStore) ──► localStorage
```

There is **no repository or service abstraction**. A page component such as `src/pages/Index.tsx:12` imports `listEventRecords` from the storage module directly. Fifteen page components do this. Introducing a real API later therefore requires editing all of them — which is exactly why the abstraction seam is priority P1 in §5.

2.3 Persistence map

Seven `localStorage` keys under **two inconsistent naming conventions**:

Key	Written by	Contains
`flyer_db_v1`	`appDatabase.ts`	users (incl. **plaintext passwords**), events, attendance, verifications
`flyer_rsvp_v1`	`checkInStore.ts`	RSVP entries
`flyer_reminders_v1`	`checkInStore.ts`	per-user reminder toggles
`flyer_checkin_tokens_v1`	`checkInStore.ts`	issued QR tokens
`flyerrr_auth`	`AuthContext.tsx`	session: `isLoggedIn` , `role` , `userId`
`flyerrr_follows`	`FollowContext.tsx`	followed org IDs
`flyerrr_theme`	`ThemeContext.tsx`	light / dark

2.4 Dependencies

- **Runtime:** React 18.3.1, react-router-dom 6.30.1, Tailwind 3.4.17, Radix UI (33 packages), TanStack Query 5.83.0, framer-motion 11, zod 3.25, react-hook-form 7.61
- **Build:** Vite 5.4.19 (Node ≥ 20 required), TypeScript 5.8.3, `lovable-tagger` 1.1.13
- **Unused but installed:** TanStack Query (provider mounted, **zero queries**), react-hook-form + zod (forms use raw `useState`), 30+ Radix packages backing the 33 unused UI components

- **Missing:** `lovable-agent-playwright-config`, imported by `playwright.config.ts` and `playwright-fixture.ts` but absent from `package.json`

---

3. Risk & Technical Debt Register

Severity = impact × likelihood. **P0** blocks the product from being real; **P1** blocks safe development; **P2** is cleanup with real cost if deferred.

🔴 P0 — Blocking

R-01 · Cross-device check-in is architecturally impossible

Evidence: `src/data/checkInStore.ts:86-101` writes issued tokens to `localStorage['flyer_checkin_tokens_v1']`. `validateCheckInToken()` (line 103) reads that same key. The student issues the token on *their* device; the officer validates on *theirs*. Two isolated stores — the lookup returns `"QR code not recognized."` 100% of the time. **Impact:** The product's headline feature cannot function outside a single-browser demo. **Fix:** Server-issued, HMAC-signed, single-use tokens validated by an API endpoint. **Estimate:** 8 pts (depends on R-04).

R-02 · Attendance is never recorded — the entire rewards chain is decorative

Evidence: `src/pages/TapIn.tsx:33-42` — `handleScannerSubmit()` validates the token, sets confirmation text, fires a toast, and **returns without writing anything**. Grep confirms no code path anywhere appends to `db.attendance`; the field is seeded at `appDatabase.ts:81` and never read or mutated again. Downstream: `AttendanceReports.tsx:9` imports the *static* `attendanceRecords` array; `AttendanceHistory.tsx:4-10` keeps a **fourth, separate** hardcoded array; `RewardsPerks.tsx:35` hardcodes `450` total points and 8 fixed badges. **Impact:** Attending an event changes nothing. Points, badges, history, reports, and the resume are all cosmetic. This is the value proposition, and it is not implemented. **Fix:** Persist `AttendanceRecord` on validated check-in; make all four screens read it. **Estimate:** 8 pts.

R-03 · Passwords stored in plaintext, readable from DevTools

Evidence: `AppUser.password: string` (`appDatabase.ts:12`) is serialized straight into `localStorage['flyer_db_v1']`; `authenticateUser()` (line 179) compares `user.password === input.password`. Every account's credentials are one DevTools panel away. **Impact:** Severe. Students reuse university passwords. This is an ethics and privacy exposure, not merely a code smell, and it cannot ship to real users. **Fix:** Move authentication server-side; hash with bcrypt/argon2; never persist credentials client-side. **Estimate:** 5 pts (depends on R-04).

R-04 · No backend exists

Evidence: Zero matches for `fetch(`, `axios`, `supabase`, or `firebase` across `src/`. The network boundary is never crossed. **Impact:** No multi-user state, no cross-device anything, no durability, no real authorization. Clearing browser data destroys the entire dataset. **Fix:** Stand up an API + Postgres. **This is the defining scope of Senior Project II. Estimate:** 13 pts.

🟠 P1 — Blocks safe development

R-05 · Authorization is client-side only and trivially bypassable

Evidence: `App.tsx:29-31` — `ProtectedRoutes` renders based on `auth.isLoggedIn`, read from `localStorage['flyerrr_auth']`. Pasting `localStorage.setItem('flyerrr_auth','{"isLoggedIn":true,"role":"org_officer"}')` into the console grants full officer privileges: the verification queue, attendance reports, and event creation. **Fix:** Server-side session validation and per-route role enforcement. **Estimate:** 5 pts.

R-06 · QR anti-fraud measures are cosmetic

Evidence: `checkInStore.ts:78` — `encodePayload = btoa(JSON.stringify(payload))`. Base64 is an encoding, not a signature. Any student can decode a token, edit `eventId`, `userId`, or `expiresAt`, and re-encode it. The 20-second TTL and the `"Screenshot codes are rejected"` check both run on data the attacker controls. **Fix:** HMAC-sign server-side; verify server-side; store nonces in the database. **Estimate:** 5 pts.

R-07 · React hooks called conditionally — crash risk

Evidence: `src/pages/EventDetail.tsx:26-32` returns early when `!event`, but two `useEffect` calls follow at lines 38 and 43. ESLint reports two `react-hooks/rules-of-hooks` **errors**. If the component instance is reused across a route change from a valid to an invalid event id, React throws "*Rendered fewer hooks than expected.*" **Fix:** Move both effects above the early return and guard inside them. **Estimate:** 1 pt.

R-08 · Effectively zero test coverage

Evidence: `src/test/example.test.ts` is the only test file, and its single assertion is `expect(true).toBe(true)`. ~7,600 lines of source have no meaningful coverage. Compounding this, `playwright.config.ts` and `playwright-fixture.ts` import `lovable-agent-playwright-config`, which is **not in** `package.json` — E2E cannot run at all. **Fix:** Add the missing dependency; unit-test the data layer first (it is pure and easy to test); add E2E for login → RSVP → check-in. **Estimate:** 8 pts.

R-09 · No CI, no branch protection

Evidence: No `.github/` directory. `actions/workflows` → `total_count: 0`. `branches/main/protection` → `404`. Nothing prevents merging a branch that fails to build. Four PRs have been merged with no automated verification. **Fix:** GitHub Actions running install → typecheck → lint → test → build on every PR; require the check before merge. **Estimate:** 3 pts.

R-10 · Broken features shipped as if complete

Evidence:

- **Saved page is permanently empty.** `Saved.tsx:8` filters `mockEvents.filter(e => e.saved)`; `saved` is `false` on all 9 seed events and the bookmark button (`EventCard.tsx:66`) only calls `setSaved()` — component state, never persisted.
- **Explore cannot find officer-created events.** `Explore.tsx:4` searches `mockEvents`, not `listEventRecords()`. Anything an officer creates is invisible to search.

- **Verification approvals do not persist.** `OrganizationVerificationQueue.tsx:18` uses local `useState`; `db.verifications` is seeded and never used.
- **Resume Builder saves nothing.** `ResumeBuilder.tsx:32` — `handleSave()` sets `editing=false` and fires a toast.

Impact: A demo that touches any of these visibly fails. Each is a small fix; together they signal that "renders" was mistaken for "works." **Estimate:** 5 pts total.

R-11 · Three competing lockfiles

Evidence: `bun.lock`, `bun.lockb`, and `package-lock.json` all committed. There is no single source of truth for dependency resolution — teammates using different package managers get different trees, and "works on my machine" becomes unfalsifiable. **Fix:** Choose one (bun is fastest here — 491 packages in 3.29 s); delete the others; document it. **Estimate:** 1 pt.

R-12 · Node version requirement is undocumented and unmet

Evidence: The repo requires Node ≥ 20 (Vite 5, ESLint 9). The audit machine's default was Node v16.15.1, on which `npx eslint .` fails outright with `ERR_MODULE_NOT_FOUND` and dozens of `EBADENGINE` warnings. There is no `.nvmrc`, no `engines` field, and no README setup instructions. **Fix:** Add `.nvmrc` (`20`), an `engines` field, and a README Getting Started section. **Estimate:** 1 pt.

R-13 · 11 dependency vulnerabilities (10 high)

Evidence: `npm audit --production` → 11 vulnerabilities. Includes a PostCSS path-traversal advisory (GHSA-r28c-9q8g-f849) and a `yaml` stack-overflow (GHSA-48c2-rrv3-qjmp). `npm audit fix` resolves them. **Estimate:** 1 pt.

🟡 P2 — Cleanup with compounding cost

R-14 · Event dates are display strings, not timestamps

Evidence: `mockEvents.ts:62` — `date: "Fri, Sep 12"`, `time: "6:00 PM"`. There is no machine-readable date anywhere in the schema. Consequences: events cannot be sorted chronologically, "upcoming vs. past" is uncomputable, reminders cannot be scheduled, and timezones are undefined. `date-fns` is installed but effectively unused for events. **Impact:** Growing — this is the schema, so every feature built on it inherits the defect. Fix it *before* the backend schema is designed, not after. **Estimate:** 3 pts.

R-15 · Substantial dead code

Evidence: `EventDetailModal.tsx` (209 lines), `FilterBar.tsx` (52), and `NavLink.tsx` (28) have **zero import sites** — 289 lines of unreferenced code. Separately, **33 of 48** shadcn/ui components are never imported (~2,900 lines), each dragging in a Radix dependency. **Estimate:** 2 pts.

R-16 · 545 KB single-chunk bundle

Evidence: `vite build` → `dist/assets/index-C75k-m_u.js 544.69 kB | gzip: 167.70 kB`, with Vite's own warning that chunks exceed 500 kB. All 14 routes are eagerly imported in `App.tsx`. For a mobile-first campus app on university Wi-Fi, this is the wrong default. **Fix:** `React.lazy()` per route + `manualChunks` for vendor code. **Estimate:** 2 pts.

R-17 · TypeScript safety is switched off

Evidence: `tsconfig.app.json` sets `strict: false`, `noImplicitAny: false`, `noUnusedLocals: false`, `noUnusedParameters: false`; `tsconfig.json` adds `strictNullChecks: false`. ESLint disables `@typescript-eslint/no-unused-vars` entirely. A passing `tsc --noEmit` therefore proves very little — null-dereference bugs like R-07 pass the typechecker. **Fix:** Enable `strict` and `strictNullChecks`; fix fallout incrementally per directory. **Estimate:** 5 pts.

R-18 · Vendor coupling to Lovable.dev

Evidence: `lovable-tagger` sits in the Vite plugin chain (`vite.config.ts:4,15`); `gpt-engineer-app[bot]` authored **11 of 26 commits (42%)**. The team did not write, and may not fully understand, a large share of the code it now owns. **Fix:** Remove the tagger once codegen stops; schedule a team code-reading session over `src/data/` and `src/contexts/` before extending them. **Estimate:** 2 pts.

R-19 · Unused architectural investments

Evidence: `QueryClientProvider` is mounted (`App.tsx:54`) with **zero queries written** — every page fetches synchronously in render. `react-hook-form` + `zod` are installed, but `Login.tsx` and `CreateEvent.tsx` use raw `useState` with no validation beyond `required`. **Impact:** Not harmful today, but these are exactly the right tools for the API migration. Adopting them *during* R-04 costs nothing extra; retrofitting later costs a rewrite. **Estimate:** 3 pts (folded into R-04).

Risk summary

Severity	Count	Total estimate
🔴 P0 — Blocking	4	34 pts
🟠 P1 — Blocks safe development	9	30 pts
🟡 P2 — Compounding cleanup	6	17 pts
Total	**19**	**81 pts**

---

4. Backlog Health Assessment

4.1 What was audited

Signal	Command	Result
GitHub Issues (all states)	`gh issue list --state all`	**0**
Open issue count	`gh api repos/oanyamele/flyerrr`	**0**
Milestones	`gh api .../milestones`	**0**
Labels	`gh api .../labels`	9 — all GitHub defaults, none project-specific
Pull requests	`gh pr list --state all`	4, all merged, **0 linked to an issue**
Project board	manual verification by team[1]	**Exists — draft/note items only**
CI workflows	`gh api .../actions/workflows`	**0**
Branch protection on `main`	`gh api .../branches/main/protection`	**404 Not Found**

[1] The board itself cannot be read via API with the team's current `gh` token — the Projects v2 API requires the `read:project` scope (`gh auth refresh -s read:project`, then re-run the query in Appendix A). Every repository-level signal in this table was re-verified via API on 2026-09-01 and is unchanged. Board contents were confirmed by direct inspection.

4.2 Assessment

The course requirement — *backlog items placed on a GitHub Project Board* — **is met in form.** A board exists and holds items. But every item is a **draft note** typed directly into a column rather than a real GitHub Issue, and that distinction is what makes the backlog unusable for Senior Project II.

A draft card is a sticky note. It cannot be assigned, labeled, estimated, linked to a pull request, closed by a commit, filtered, or referenced from code review. The evidence of the consequence is already in the repo: **four merged pull requests, none of which references an issue.** There is no trace from "we decided to build this" to "here is the code that built it." With four contributors, that traceability is the difference between a team and four people editing the same folder.

4.3 Scored rubric

Criterion	Score	Finding
Items exist on a board	🟢 4/5	Board is populated — the requirement is satisfied
Items are real GitHub Issues	🔴 0/5	Draft notes only; 0 Issues exist in the repository
Acceptance criteria defined	🔴 0/5	Draft cards carry a title and nothing else
Estimated / sized	🔴 0/5	No estimate field, no story points
Assigned to an owner	🔴 0/5	Draft items cannot hold assignees
Labeled / categorized	🔴 1/5	Only the 9 default GitHub labels; none applied
Prioritized	🟠 2/5	Column position implies order; no explicit priority field
Linked to PRs / commits	🔴 0/5	4 of 4 PRs unlinked
Grouped into milestones/sprints	🔴 0/5	0 milestones
Reflects known technical debt	🔴 0/5	None of the 19 risks in §3 appears on the board
Total	🔴 **7 / 50 (14%)**	**Present but not yet a working backlog**

4.4 Remediation — resources provided

Three artifacts accompany this report to close the gap:

File	Purpose
`docs/backlog.csv`	29 backlog items — title, body, labels, priority, estimate, epic, sprint
`scripts/seed-backlog.sh`	Creates the labels, milestones, and all 29 Issues via `gh`, then adds them to the Project Board
`.github/workflows/ci.yml`	CI gate closing R-09

Every backlog item traces to a specific finding in §3, so the board becomes the execution plan for this report rather than a parallel document.

Target state by end of Sprint 1: ≥ 40/50 on the rubric above — all items as real Issues with acceptance criteria, estimates, owners, labels, and milestone assignment.

---

5. Initial Senior Project II Priorities

Guiding principle

> **Stop adding screens. Make the screens that exist tell the truth.**

flyerrr already looks finished. The work ahead is to make it *be* what it appears to be. Every P0 in §3 traces to one root cause — R-04, the missing backend — so that is where Sprint 1 goes.

Sprint 0 — Foundation (Week 1) · 12 pts

Cheap, unblocks everyone, do it first.

#	Item	Closes	Pts
1	Convert all draft board items to real Issues; apply labels, estimates, owners, milestones	§4	3
2	Add CI workflow: install → typecheck → lint → test → build; require on PR	R-09	3
3	Fix conditional hooks in `EventDetail.tsx`	R-07	1
4	Add `.nvmrc`, `engines`, and a real README with setup steps	R-12	1
5	Delete two of three lockfiles; standardize on bun	R-11	1
6	Run `npm audit fix`; add the missing Playwright dependency	R-13, R-08	1
7	Enable branch protection on `main`	R-09	1
8	Delete 289 lines of dead components + unused shadcn files	R-15	1

Exit criterion: a green CI badge on `main`, and a board where every card is an estimated, assigned Issue.

Sprint 1 — Backend & the repository seam (Weeks 2–4) · 28 pts

The defining work of the semester.

#	Item	Closes	Pts
9	Record an ADR for the backend stack — timeboxed to Week 2	R-04	2
10	Design the relational schema — **with ISO timestamps for events** (fix R-14 *before* the schema is set)	R-04, R-14	3
11	Stand up API + Postgres; deploy a staging environment	R-04	8
12	Introduce the repository interface; keep localStorage as a test/offline adapter	R-04	5
13	Server-side auth: hashed passwords, session tokens, real role enforcement	R-03, R-05	5
14	Migrate the events feed to TanStack Query hooks behind the repository	R-19	5

Exit criterion: two students on two different laptops see the same event created by an officer on a third.

Sprint 2 — Make the core loop real (Weeks 5–7) · 23 pts

#	Item	Closes	Pts
15	Server-issued HMAC-signed, single-use check-in tokens	R-01, R-06	8
16	Persist `AttendanceRecord` on validated check-in	R-02	5
17	Rewire Reports, History, Rewards, and Resume Builder to live attendance data	R-02	5
18	Persist saves/bookmarks; point Explore at the database	R-10	3
19	Persist organization verification decisions	R-10	2

Exit criterion: a student checks in on their phone, an officer scans on theirs, and the student's points increase on a third device. *This is the demo the project is judged on.*

Sprint 3 — Hardening (Weeks 8–10) · 35 pts

Larger than the others by design — it absorbs every P2 item. Trim from the bottom if Sprints 1–2 run long; the first two rows are non-negotiable.

#	Item	Closes	Pts
20	Unit-test the data layer to ≥ 60% coverage	R-08	8
21	E2E: login → RSVP → check-in → points awarded	R-08	5
22	Enable `strict` + `strictNullChecks`; fix fallout	R-17	5
23	Adopt react-hook-form + zod on all forms	R-19	3
24	Add empty, loading, and error states across all screens	R-02, R-10	3
25	Accessibility pass: keyboard nav, labels, contrast	known unknown	3
26	Route-level code splitting; get the initial chunk under 250 KB	R-16	2
27	Remove `lovable-tagger`; team code-reading session over `src/data/`	R-18	2
28	Unify localStorage key naming; add schema versioning	§2.3	2
29	Decide: implement Web NFC, or remove the simulated tap	R-10	2

Capacity summary

Sprint	Items	Points
Sprint 0 — Foundation	8	12
Sprint 1 — Backend & Seam	6	28
Sprint 2 — Core Loop	5	23
Sprint 3 — Hardening	10	35
Total	**29**	**98**

The backlog (98 pts) exceeds the risk register (81 pts) because it also carries process and UX work — CI, ADRs, accessibility, empty states — that is not itself a *risk* but is required to close one.

What is explicitly *not* a priority

Deferring these is a decision, not an oversight:

- **New features.** No new screens until Sprint 2 exits. There are already four screens displaying data that does not exist.
- **NFC.** `TapIn.tsx:130` is a "Simulate NFC Tap" button. Real Web NFC is Chrome-on-Android only and cannot carry an iOS demo. Make the QR path work first (R-01); revisit NFC only if Sprint 2 finishes early.
- **Push notifications.** `NotificationSettings.tsx` renders toggles that control nothing. Real push requires the backend from Sprint 1 — it is downstream by definition.
- **Visual redesign.** The UI is the strongest part of this project. Leave it alone.

Team risks to manage

Risk	Mitigation
42% of commits are bot-authored; team ownership of `src/data/` and `src/contexts/` is unproven	Sprint 3 code-reading session; require a human reviewer on every PR touching those directories
Four contributors, zero linked PRs, no branch protection	Sprint 0 items 2 and 7 make this structural rather than a matter of discipline
Backend is new scope for the team	Timebox the stack decision to Week 2; prefer boring and well-documented (Postgres + a managed host) over novel
No one currently owns the schema	Assign an owner in Sprint 0; item 9 is the highest-leverage decision of the semester

---

6. Conclusion

flyerrr enters Senior Project II with a genuine asset and a genuine debt. The asset is a polished, coherent, mobile-first interface across 14 routes and two user roles, built on a clean design-token system. The debt is that essentially none of it is connected to anything.

The audit found **19 tracked risks (81 story points)**, of which four are blocking, and a project board that satisfies the letter of the backlog requirement while providing none of its value. Both are addressable — the backlog gap in one sprint, the architecture gap in three — and the accompanying `scripts/seed-backlog.sh` converts this report into 29 tracked, estimated Issues on the board in a single command.

The semester's success criterion is a single sentence: **a student checks in on one device and an officer verifies it on another.** Every priority in §5 exists to serve it.

Appendix A — Reproducing this audit

```
git clone https://github.com/oanyamele/flyerrr.git && cd flyerrr
nvm use 20            # Node ≥ 20 is required; the build fails on Node 16
bun install
npx tsc --noEmit -p tsconfig.app.json
npm run build && npm test && npx eslint .
npm audit --production
gh issue list --state all
gh api repos/oanyamele/flyerrr/actions/workflows --jq '.total_count'
```

Appendix B — Verified metrics

Metric	Value
Total source lines (`src/`)	≈ 7,623
Source files	80
Routes	14
Commits (all branches)	26
Commits authored by `gpt-engineer-app[bot]`	11 (42%)
Contributors	4 humans + 1 bot
Merged pull requests	4
GitHub Issues	0
CI workflows	0
Test files / meaningful assertions	1 / 0
ESLint errors / warnings	5 / 11
Dependency vulnerabilities	11 (10 high, 1 moderate)
Production bundle (JS)	544.69 kB (167.70 kB gzip)
Build time	2.09 s
Unused shadcn/ui components	33 of 48
Unreferenced component files	3 (289 lines)
`fetch()` calls in the codebase	**0**

Appendix C — Architecture diagram source

See `docs/architecture.md`. Diagrams are authored in Mermaid and version-controlled alongside the code, per the Swark workflow; a timestamped copy is kept in `swark-output/`.